

Decoding Crime Narratives using NLP and Big Data Analytics

Abstract

Unstructured incident narratives contain descriptions of events, participants, places and objects that are difficult to compare with simple database queries. This project implements a reproducible system for converting such narratives into searchable, annotated and measurable representations. A seeded synthetic corpus supplies 60,000 fictional reports, exact entity offsets and scenario-level train/validation/test assignments. Apache Spark performs distributed transformations, data-quality checks, deduplication, aggregation, Parquet storage and an MLlib baseline. Python models compare sparse word representations with neural embeddings and a fine-tuned BERT transformer. A linear-chain conditional random field extracts entity roles, while latent Dirichlet allocation discovers recurring language. A Streamlit interface exposes trends, retrieval, parsing, sentiment, summaries and evaluation. The project demonstrates engineering and NLP methods; it does not establish forensic validity or real-world crime trends.

1. Problem and objectives

The input is a collection of text-based case narratives. The outputs are a cleaned corpus, narrative category predictions, contextual entity spans, recurring topics, linguistic annotations, extractive summaries, and an interactive research interface.

The research questions are: How do lexical and neural representations compare under a fixed evaluation protocol? Can sequence labeling distinguish the alleged suspect and reporting victim from contextual cues? Which themes appear in unsupervised word co-occurrence patterns? How can distributed processing feed a transparent user-facing NLP application?

The system operates in English and accepts pasted text or UTF-8 case files for live inference. It performs document classification, sequence tagging, sentiment analysis, parsing, summarization and information retrieval. It does not identify real people, determine guilt, or estimate actual danger.

2. Dataset and provenance

The corpus is generated by `crime_nlp/generate.py` using seed 42. Its eight balanced source categories are Arson, Assault, Burglary, Cybercrime, Fraud, Robbery, Theft and Vandalism. Dates span 2023–2025, and locations belong to six fictional districts. Names are formed from an invented name pool; no real case files are used. The generator varies dates, times, items, participants, locations, weapon mentions, evidence statements, and extended follow-up paragraphs. This produces at least 50,000 distinct narratives while keeping provenance auditable.

The canonical build includes 60,000 valid reports plus 1,500 duplicate rows, 600 null narratives, 300 whitespace-only narratives, and 300 malformed dates. These errors exercise the cleaning pipeline. Metadata includes report ID, source category, date, district, scenario template group, split, source marker and gold entity offsets. The category, split, template ID and gold annotations are never passed as classification input features.

Each crime has eight scenario templates. Six are assigned to training, one to validation, and one to testing before model fitting. All reports from a template remain together. The resulting held-out evaluation is harder than a random row split, but vocabulary and generator conventions are still shared. The corpus cannot represent dialects, ambiguous cases, reporting biases, legal distinctions, or actual population distributions.

3. Big Data architecture

The Spark driver reads JSONL with an explicit schema and repartitions it into eight partitions. Transformations remove nulls, normalize a separate text column, reject blank normalized text and invalid dates, compute a SHA-256 normalized-content hash, and retain a deterministic canonical record per hash. Original text is preserved because entity offsets and syntax depend on original characters. A window function selects the canonical duplicate survivor by report ID.

The distinction between transformations and actions is visible in the source. Filtering, column construction, repartitioning and grouping construct lazy plans. Counts, fitting, Parquet writes and the small aggregate conversion trigger execution. Spark SQL groups the cleaned data by month, crime type and district. Parquet is partitioned by split for downstream training and auditability. Only a small trend aggregate is collected through `toPandas` during the Spark stage.

The verified execution uses Spark 3.5.5 in `local[2]` mode with eight shuffle partitions. Multiple Spark tasks execute on two local cores; this is not proof of horizontal scaling across machines. The same code accepts a standalone Spark master URL, and container configuration includes a master, worker and driver service. That deployment path is provided but was not executed on the build computer.

In the Hadoop syllabus context, HDFS supplies distributed storage and replication, YARN can manage cluster resources, and MapReduce expresses map/shuffle/reduce jobs. This project uses Spark's DAG execution and Hadoop-compatible filesystem interfaces rather than launching a Hadoop MapReduce job. It stores data on the local filesystem during the verified run. HDFS, YARN and Google File System are architectural discussion topics here, not services claimed to be running.

The separate Scala program exercises typed Datasets, immutable case classes, a text-transformation trait, pattern matching, closures, lists, sets, maps, `map`, `flatMap`, `filter`, grouping and SQL. It compiles with the Scala compiler included in PySpark's distribution. It reads the same Parquet output and computes category families and word frequencies.

4. Linguistic processing

spaCy supplies sentence boundaries, tokenization, part-of-speech tags, fine-grained tags, lemmas, morphological features and dependency heads. In the dashboard, users select a sentence and inspect a dependency graph alongside a token table. Lemmatization illustrates morphological normalization, while dependency relations show syntactic relationships such as subject, object and modifier.

The classification pipelines tokenize their own textual input. Morphological analysis is exposed separately instead of silently modifying the original narrative. Gold CRF labels are aligned to a deterministic offset-aware tokenizer; round-trip tests ensure annotation coordinates reconstruct the original spans exactly.

5. Word representation comparison

All four primary classifiers use the same 4,800 training narratives and 1,200 test narratives, stratified by category within the predefined split. There are 800 validation narratives. Split IDs are exported for auditing.

Bag of Words: a count vectorizer records unigram and bigram frequencies and retains up to 12,000 features. A multinomial logistic regression maps the sparse document vector to a category.

TF-IDF: a vectorizer combines sublinear term frequency with inverse document frequency, using the same vocabulary limit and n-gram range. A separate logistic regression is trained. Neither vectorizer observes validation or test documents during fitting.

Neural Word2Vec: Gensim trains a 100-dimensional skip-gram neural embedding model with negative sampling on training narratives. It uses a five-token context window, minimum count two, twelve epochs, one worker, a fixed seed, and stable hashing. Document vectors average in-vocabulary word embeddings, followed by logistic regression. Averaging loses word order, making this a useful contrast with BERT rather than a contextual sequence model.

BERT: the pretrained `google/bert_uncased_L-2_H-128_A-2` checkpoint contains two transformer layers with 128 hidden units and two attention heads. A new eight-class classification head is trained along with all encoder layers for three epochs using AdamW. Batches contain 32 narratives, input is capped at 160 subword tokens, and gradients are clipped. The saved checkpoint is chosen by validation macro-F1; the test set is not used to select it. Long narratives are truncated for BERT classification; entity extraction, syntax and summaries operate on the full accepted input.

The project does not assume the neural model must win. Measured outcomes are retained even when simpler models perform better. The primary model comparison is displayed independently from the full-corpus Spark MLlib baseline.

6. Sequence labeling and topic modeling

A linear-chain CRF predicts BIO tags for SUSPECT, VICTIM, LOCATION and WEAPON. Observations include the token's lowercased form, suffixes, capitalization, digits, punctuation and neighboring-token context. CRFs jointly score sequences using observation and transition features, supporting multi-token role spans. The implementation trains on 2,400 training narratives and evaluates 600 held-out narratives. Strict entity F1 requires both a matching span and matching label. Dashboard role highlighting uses model predictions, not the generator's labels.

The transformer performs narrative sequence classification; the CRF performs sequence labeling. They are complementary tasks rather than a single joint BERT-CRF model. The chosen sequence-model syllabus option is CRF; HMM is not additionally required or implemented.

An initial CRF with only a two-token context window failed to distinguish participant roles on the unfamiliar test sentence frames. Sentence-level role-cue observations were added, and candidate selection used validation F1. The initial test result and validation comparison are preserved in separate artifacts. This revision was informed by initial test error analysis, so the revised score is labeled a development evaluation rather than an untouched final benchmark. Role cues are input features learned by the CRF; no gold entities or post-hoc rule correction enter the predictions.

LDA is trained on training-set count vectors to discover eight latent topics. Top terms, held-out perplexity, top-term diversity and corpus topic assignments are saved. These are co-occurrence patterns and may reflect reporting style or names as well as incident themes. Topic IDs have no intrinsic meaning, and the system does not equate them automatically with verified *modus operandi*. LDA is unsupervised; gold crime categories are not supplied to it.

7. Applications and interface

Sentiment: NLTK VADER computes positive, neutral, negative and compound polarity. Crime narratives often contain negative vocabulary independent of actual immediacy or severity. Therefore, a separate rule-based threat-language score counts explicit cues and suppresses common local negations. The score is capped at twelve and displayed as Low, Moderate or High language intensity. It is a transparent teaching heuristic, not a validated risk estimator, and its negation handling is limited to short clause-local contexts.

Summarization: sentences become TF-IDF vectors and a cosine-similarity graph. PageRank assigns centrality scores; a small lead-sentence prior anchors the described event. The top two sentences are returned in original order. This TextRank variant is extractive and creates no new factual claims, although source selection can still omit important qualifications.

Retrieval: corpus search applies the training-fitted TF-IDF vectorizer to the query and scores it against a sparse matrix aligned with dashboard row order. Category, district and date filters constrain results. Report IDs can be searched directly. Within-case evidence retrieval returns the most similar source sentence and abstains when lexical overlap is too weak. This is evidence search, not a generative question-answering model.

Dashboard: Overview shows trends, category composition, district/category heatmaps and polarity distribution. Case explorer supports retrieval and detailed inspection. Narrative lab accepts text and optional BERT inference. Model evaluation displays actual scores, confusion matrices, per-class metrics, strict CRF results and training history. Topics shows LDA terms and examples. Pipeline & syllabus exposes cleaning counts and the implementation mapping.

Uploaded narratives are processed in memory and held in the current Streamlit session; the application does not write them to disk. User text is escaped before HTML highlighting. Input length is limited to 30,000 characters and uploaded files to 120 KB. The default server binds only to the local interface.

8. Evaluation and results

The generated `docs/RESULTS.md` and `artifacts/evaluation.json` contain results from the executed pipeline. Accuracy is correct predictions divided by test examples. Macro-F1 averages class F1 values equally; weighted F1 weights each class by support. Confusion matrices reveal cross-class errors. CRF strict span F1 evaluates complete entities rather than counting the many outside tokens as successes. LDA perplexity and top-term diversity characterize the topic model without asserting semantic correctness.

The Spark MLlib baseline uses `RegexTokenizer` → `StopWordsRemover` → `HashingTF` → `IDF` → `LogisticRegression`. It fits the full training split and predicts the full test split. Its weighted F1 and accuracy have a different sample size than the bounded four-model comparison and are labeled accordingly.

VADER polarity, threat-language rules, syntax parsing and summaries lack a separately annotated gold evaluation corpus in this project. Accordingly, no accuracy, danger-detection performance or summary ROUGE score is invented for them. Summary tests verify extractiveness and length, not clinical or forensic adequacy.

Tests cover all 64 template groups, character offsets, BIO reconstruction, clause-local negation, summary grounding, evidence abstention, HTML escaping, dataset quality, split leakage, index alignment, metric consistency, offline BERT inference, and all dashboard pages. Runtime manifests and logs preserve execution evidence.

9. Limitations and future work

Synthetic narrative templates constrain linguistic variety and can inflate performance. Category boundaries may be ambiguous in genuine records, while the generator assigns one source label per report. English-only pretrained models and simple token features may generalize poorly across communities and writing styles. CRF role extraction can confuse mentions in unfamiliar sentence structures. Topic terms may capture names or template artifacts. Sentiment is not a threat assessment. Extractive summaries need human review, and model probabilities are uncalibrated.

The NLP training stage is bounded and local; the full corpus is enriched in Python batches. The app keeps its dataset and sparse index in memory. Future scaling should introduce distributed inference, partition-aware reads, a retrieval service, shared storage, artifact versioning and a task queue. A real-data study would require appropriately licensed, de-identified records, independent human annotations, external validation, error analysis across writing styles, calibration and a redesigned deployment review.

References

1. [Apache Spark installation and Java compatibility.](#)
2. [Apache Spark RDD programming guide.](#)
3. [Spark MLlib and Scala API documentation.](#)
4. [Devlin et al., BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.](#)
5. [Google small BERT model card.](#)
6. [Mikolov et al., Efficient Estimation of Word Representations in Vector Space.](#)
7. [Gensim Word2Vec documentation.](#)
8. [sklearn-crfsuite tutorial.](#)
9. [spaCy dependency and entity visualizers.](#)
10. [NLTK VADER API.](#)
11. [Mihalcea and Tarau, TextRank: Bringing Order into Text.](#)

Executed project results

Generated from saved artifacts; all scores below are measured, not illustrative.

Data and execution

- Raw reports: **62,700**
- Clean unique reports: **60,000**
- Invalid rows removed: **1,200**
- Duplicates removed: **1,500**
- Spark: **3.5.5**, master **local[2]**, **8** input partitions
- Enriched narratives: **60,000**

Same-split model comparison

Train / validation / test: 4,800 / 800 / 1,200.

Model	Accuracy	Macro F1	Weighted F1
Bag of Words	0.9175	0.9116	0.9116
TF-IDF	0.8458	0.8005	0.8005
Word2Vec	0.9075	0.9055	0.9055
BERT	0.8367	0.8143	0.8143

Entity extraction

Revised CRF strict entity F1: **0.8235** on 600 narratives.

Entity	Precision	Recall	F1
SUSPECT	1.0000	1.0000	1.0000
VICTIM	0.0000	0.0000	0.0000
LOCATION	1.0000	1.0000	1.0000
WEAPON	1.0000	1.0000	1.0000

Original CRF test F1: 0.4000. Validation F1 changed from 0.6982 to 1.0000 after adding sentence-level observations.

Initial test role errors informed feature development; the revised test score is not an untouched final benchmark.

Material limitation: held-out extraction remains weak for VICTIM. Review predicted roles against the original text; model completeness is not evidence of reliable real-world extraction.

Distributed baseline and topics

MLlib accuracy: **0.8773**; weighted F1: **0.8396**. It uses 45,008 training and 7,496 test rows, so sample sizes differ from the four-model comparison.

LDA held-out perplexity: **238.40**; top-term diversity: **0.675**.

Verification

Pytest: **17 passed in 9.65s**.

Scala report count verified: **60,000**.

The Windows Scala run completed its computations and exited successfully, with a non-fatal Spark temporary-JAR cleanup warning at shutdown.

Browser smoke check: **passed**. Pages visited: Overview, Case explorer, Model evaluation, Topics.

Interpretation

The corpus is synthetic. Scenario groups are separated, but vocabulary and generator conventions are shared. Revised CRF test results are development evaluation. No human-reference accuracy is claimed for syntax, sentiment, threat heuristics or summarization. Spark ran locally; Docker/multi-host deployment was not exercised.

Resources: `artifacts/evaluation.json` , `ner_evaluation.json` , `spark_metrics.json` , `topics.json` , split ID CSVs, and `logs/` .